

CS & IT ENGINEERING

DATA STRUCTURE

Hashing



JOIN TELEGRAM TO ENROLL

SHAIENDRA SIR

Hashing

- ✓ Hashing
- ✓ Hash function
- ✓ Collision Resolution Technique
- ✓ Linear Probing
- ✓ Quadratic Probing
- ✓ Double Hashing
- ✓ PYQ

Hashing in Data Structures

Searching Technique. [Goal of Hashing $\Rightarrow O(1)$]

Hashing in data structures is a technique used to store and retrieve data efficiently using a hash function. It is commonly used in hash tables (hash maps) to allow fast access to data in $O(1)$ average time complexity.

Unsorted Array = $O(n)$

Sorted Array = $O(\log n)$

Linked list = $O(n)$

Binary search Tree $\begin{cases} O(\log n) \\ O(n) \end{cases}$

AVL - $O(\log n)$

Hashing $\Rightarrow O(1)$

Still worst case = $O(n)$

$\Rightarrow$ worst case

The following components are used in Hashing

1) KeySpace(n)

2) Hash function

3) Hash Table.

$\Rightarrow$ worst case search $\Rightarrow O(1)$ [99%]

$O(n)$

1%

Hash function:

$$HF(key) = key \bmod m$$

$$\text{let } m = 10$$

$$key_1 = \underline{783456} \Rightarrow H(783456) = 783456 \% 10 \Rightarrow \textcircled{6}$$

↑
Hash Address

$$key_2 = \underline{56249234}$$

$$\Rightarrow H(key_2) = 56249234 \% 10$$

$$= \textcircled{4}$$

Hash Address

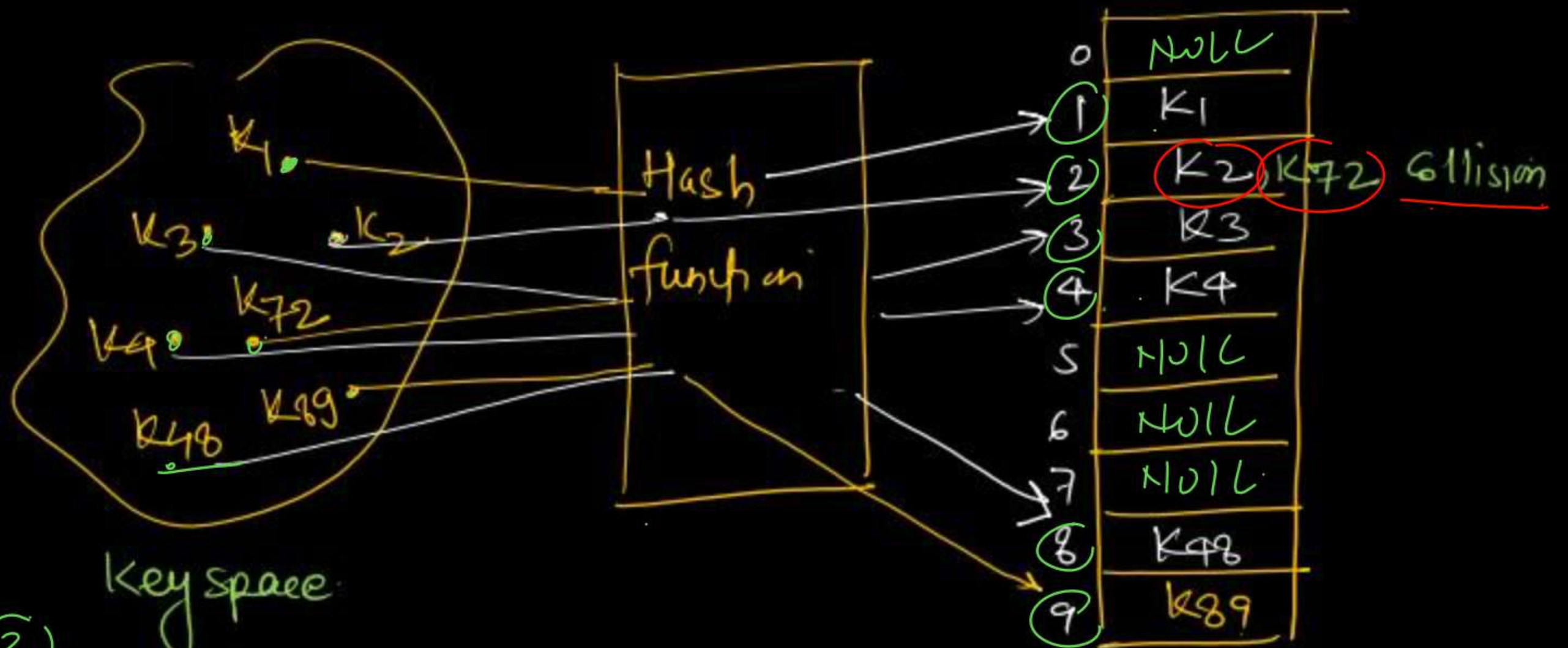
Hash function:-
 $\xrightarrow{x} \xrightarrow{x}$

$$K_2 = 2$$

$$2 \div 10 = (2)$$

$$K_{72} = 72$$

$$72 \div 10 = (2)$$



HashTable(m)

⇒ The main problem with the hash functions, if both the keys are mapped down to same index in the hash table then it is not possible to store (Collision)

⇒ To Handle the Collisions the various Collision resolution techniques are used.

Hash function:-

$$HF(Key) = Key \bmod m$$

1) Division modulo method:- do not choose m value as exactly power of 2 because if $m = 2^k$, then Hash function of Key will always LSB, k bits

$$HF(Key) = \text{LSB } k \text{ bits always}$$

Ex:- $m = 8 = 2^3 = 2^k$, $k = 3 \text{ bits}$

- 1) Key = 86 = ^{MSB}1010^{LSB}110, 86 mod 8 $\Rightarrow$? 6
2) Key = 102 = 1100110, 102 mod 8 $\Rightarrow$? 6
3) Key = 118 = 1110110, 118 mod 8 $\Rightarrow$? 6
4) Key = 126 = 1111110, 126 mod 8 $\Rightarrow$? 6
- } Collision

if $m = 7 \neq 2^3 \neq 2^k$

- 1) 86 mod 7 = ? 2
2) 102 mod 7 = ? 4
3) 118 mod 7 = ? 6
4) 126 mod 7 = ? 0
- } Unique

No Collision

NOTE $m = ?$

2) Mid Square method:-

$m = 1000$ (0 to 999)

Key = 786

$$(key)^2 = (786)^2 = 617796$$

- It is good Hash function
- Counter Example is difficult to generate

0	
177	786
999	

HashTable

$$(786)^2 = 617796$$

177

3) Digit Extraction method:-

$$\begin{cases} \text{Key}_1 = 73681426 \Rightarrow 846 \\ \text{Key}_2 = 52882466 \Rightarrow 846 \end{cases}$$

- It is Bad Hash function
- Counter Example is easy to generate.

0	
1	
846	73681426
999	

4) Folding method:-

i) Fold Boundary : Key₁ = 734 68 143

X 734
 143

 877

Key₂ = 143 68 734

 143
 734

 877

<u>877</u>	<u>73468143</u>

1a) Fold Shifting:

143
468
073

684

Key₁ = 734 68 143

Key₂ = 73 143 468

684	73468143

Hashing:- mapping larger set to smaller set (many to one)

- Searching Technique. $\approx O(1)$

$$h(U) = \{0, 1, 2, \dots, m-1\}$$

Problem:- Collision

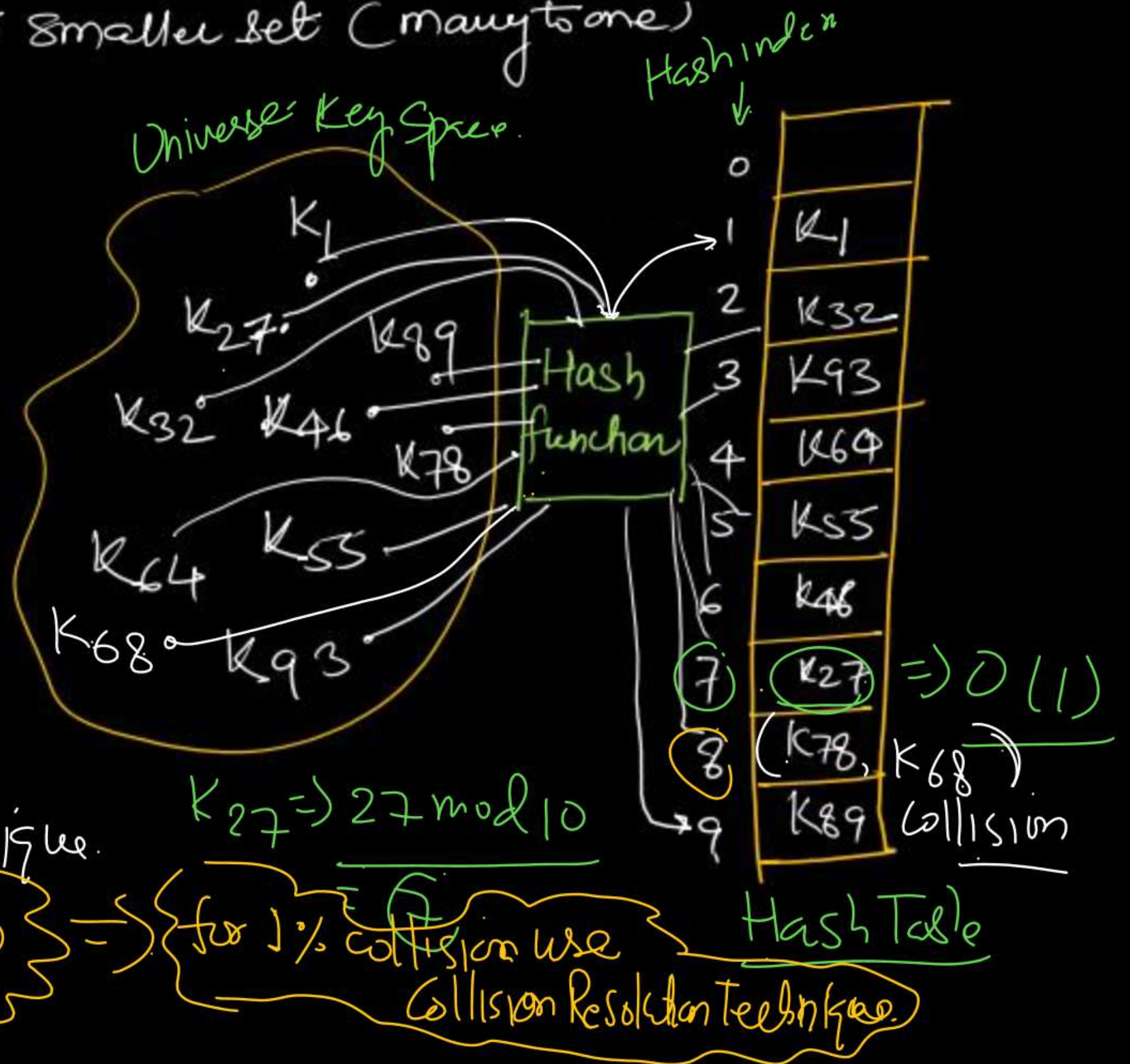
How to remove Collision:- ?

- 1) Use better Hash function but never say no collision
- 2) Use Collision Resolution Technique.

1st select Best Hash function

99% No Collision

for 1% collision use Collision Resolution Technique.



Collision Resolution Technique:- (CRT)



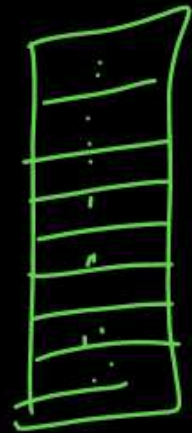
1) Chaining (outside Table) (Open Hashing)



- ✓ Key will be stored outside Hash Table
- ✓ Deletion is easy as compared to open Addressing
- ✓ No overflow
- ✓ Insertion from beginning always

2) Open Addressing (inside Table or Closed Hashing)

- ✓ Key will be stored inside Hash Table
 - 1) Linear Probing (Suffer Primary clustering)
 - 2) Quadratic Probing (Suffer Secondary clustering)
 - 3) Double Hashing
 - Deletion Complex
 - overflow occur

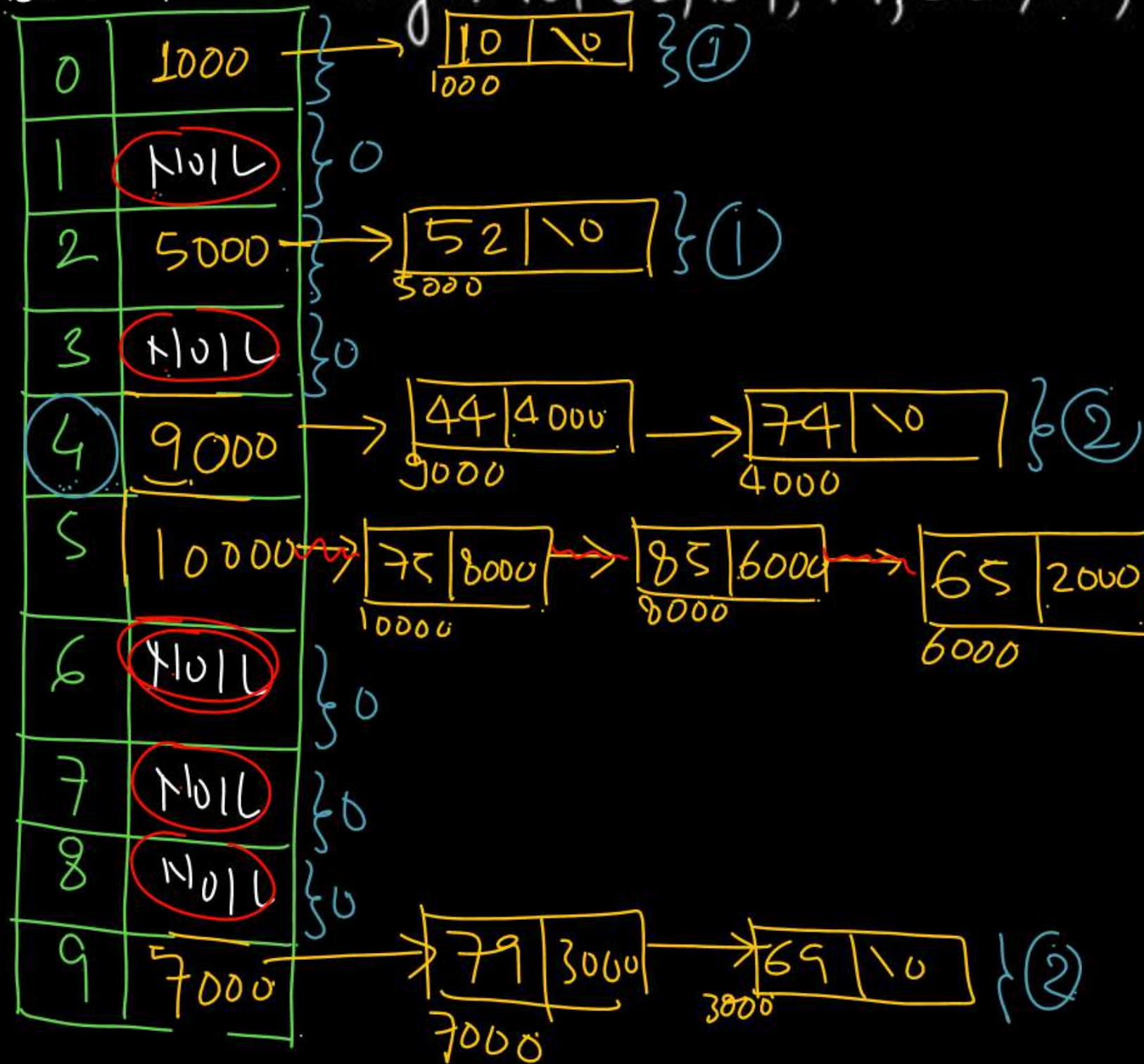


Chaining:-

Let $m = 10 (0-9)$, $H(K) = K \bmod m$

Hash Table

Keys: 10, 55, 69, 74, 52, 65, 79, 85, 44, 75



Longest chain length = 4
minimum chain length = 0

Avg chain length = $\frac{1+0+1+0+2+4+0+0+0+2}{10} = 1$

NOTE:-

- 1) The main advantage of Chaining is we can handle any no. of collisions (No overflow)
- 2) Even though the space is available inside the hash Table we are using the space outside Hash Table
- 3) Longest chain possible with n -keys is $n \Rightarrow$ so worst case = $O(n)$
- 4) Insertion, Deletion can be done at any time without affecting others
- 5) Less sensitive to the hash function or load factor
- 6) Chaining takes more space but less time

$$\text{Time} \propto \frac{1}{\text{Space}}$$

Load factor (α):- The no. of keys getting stored in 1 slot is called as load factor

$$\text{load factor } (\alpha) = \frac{n}{m} = \text{no. of elements in each slot}$$

no. of keys $\rightarrow$ n
 $\uparrow$
Size of Hash Table $\rightarrow$ m
maximum load allocated to each slot $\rightarrow$ α

$$n = k \cdot m$$

$$\begin{aligned} \text{Avg Search Time} &= \Theta(1 + \alpha) \Rightarrow \Theta\left(1 + \frac{n}{m}\right) \\ &= \Theta(1 + k) \\ &= \boxed{\Theta(1)} \end{aligned}$$

$$\alpha = \frac{n}{m} = \frac{k \cdot m}{m}$$

$$\alpha = k = \underline{\text{constant}}$$

PYQ:

Consider a hash table with 9 slots. The hash function is $h(k) = k \bmod 9$. The collisions are resolved by chaining. The following 9 keys are inserted in the order: 5, 28, 19, 15, 20, 33, 12, 17, 10. The maximum, minimum, and average chain lengths in the hash table, respectively, are

H.W

Open Addressing :- (closed Hashing)

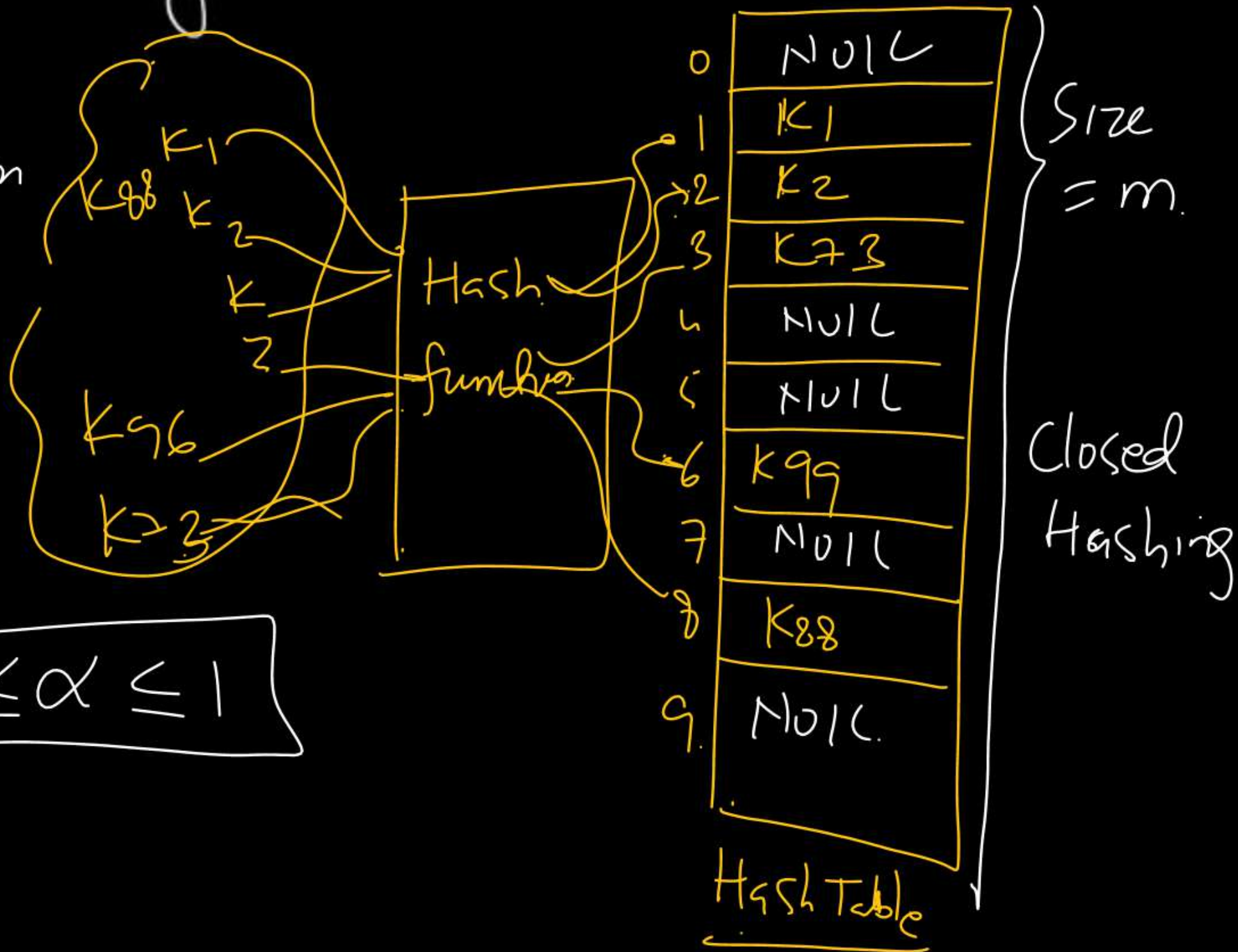
If Size of Hash Table = m
We can not insert more than m
Keys inside the Table.

$$\alpha = \frac{n}{m}$$

$$\alpha_{\max} = \frac{m}{m} = 1$$

$$0 \leq \alpha \leq 1$$

- 1) Linear Probing
- 2) Quadratic Probing
- 3) Double Hashing



Linear Probing:-

$$m = 10 \text{ (0-9)}$$

$$HFC(key) = key \bmod m$$

$$LP(20,0) = 0 \text{ Collision}$$

$$LP(20,1) = 1 \text{ Collision}$$

Hash Table

Total Collision = 9

$$LP(key, i) = (HFC(key) + i) \bmod m, \text{ for } i = 0, 1, 2, 3, \dots, m-1$$

Ex: Keys: 25, 38, 43, 68, 79, 46, 58, 65, 20

$$LP(20,2) = 2 \checkmark$$

$$LP(25,0) = (HFC(25) + 0) \bmod 10$$

$$= (25 \bmod 10 + 0) \bmod 10$$

$$= (5 + 0) \bmod 10 \Rightarrow 5$$

$$LP(65,0) = 5 \text{ Collision}$$

$$LP(65,1) = 6 \text{ Collision}$$

$$LP(65,2) = 7 \checkmark$$

$$LP(38,0) = 8 \text{ } \left. \begin{array}{l} LP(68,0) = 8 \\ LP(68,1) = 9 \end{array} \right\} \text{ Collision}$$

$$LP(68,1) = 9$$

$$LP(79,0) = 9$$

$$LP(79,1) = 10 \bmod 10 = 0$$

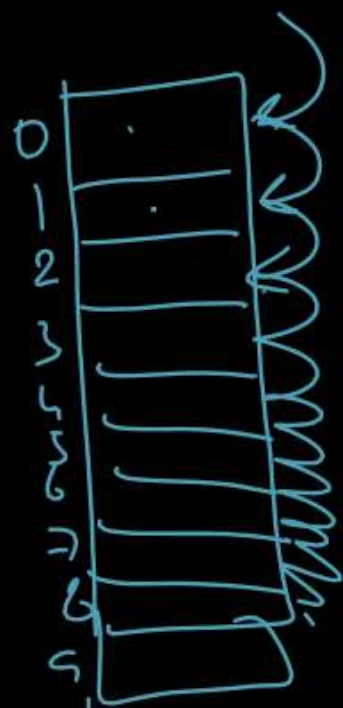
$$LP(58,0) = 8 \text{ Collision}$$

$$LP(58,1) = 9 \text{ Collision}$$

$$LP(58,2) = 0 \text{ Collision}$$

$$LP(58,3) = 1 \checkmark$$

0	79
1	58
2	20
3	43
4	NULL
5	25
6	46
7	65
8	38
9	68



Ex! - Keys(n) = ~~61~~, ~~30~~, ~~52~~, ~~43~~, ~~80~~, ~~90~~, ~~74~~, ~~82~~

$$H(K) = K \bmod m, \quad m = 10 \quad (0-9)$$

$$LP(K, i) = (H(K) + i) \bmod m$$

$$LP(61, 0) = 1$$

$$LP(80, 4) = 4$$

$$LP(30, 0) = 0$$

$$LP(90, 0) =$$

$$LP(52, 0) = 2$$

Keys $\rightarrow$ 10, 20, 30, 40, 50, 60, 70, 80,

$$LP(43, 0) = 3$$

90, 100

$$LP(80, 0) = 0 \text{ Collision}$$

$$LP(80, 1) = 1 \text{ Collision}$$

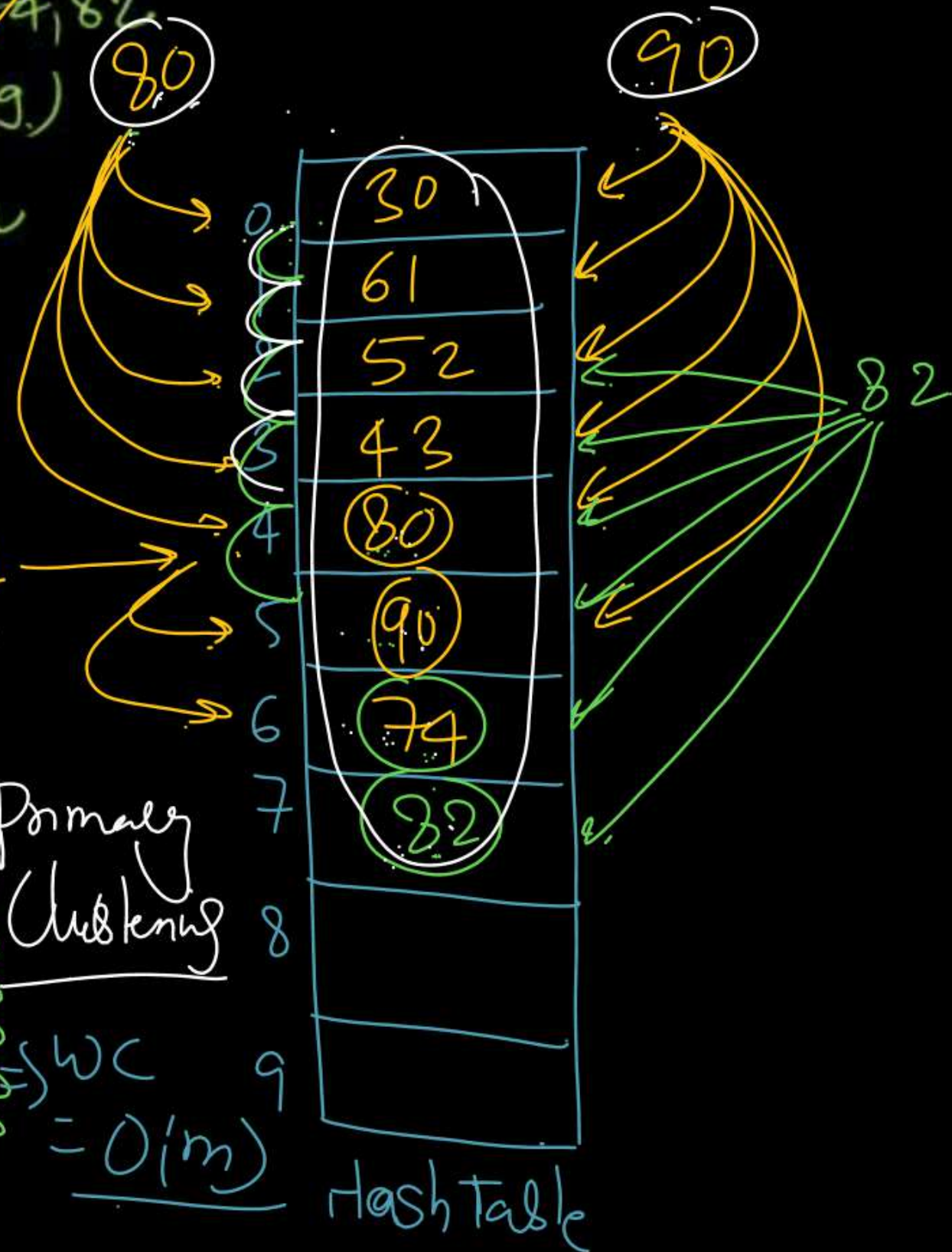
$$LP(80, 2) = 2 \text{ Collision}$$

$$LP(80, 3) = 3 \text{ Collision}$$

0	10
1	20
2	30
3	40
4	50
5	60
6	70
7	80
8	90
9	100

Primary
Clustering

$$SWC = 0/m$$



Ex! - Keys(n) = ~~61~~, ~~30~~, ~~52~~, ~~43~~, ~~80~~, ~~90~~, ~~74~~, ~~82~~

$$H(K) = K \bmod m, \quad m = 10 (0-9)$$

$$LP(K, i) = (H(K) + i) \bmod m$$

$$LP(61, 0) = 1$$

$$LP(80, 4) = 4$$

$$LP(30, 0) = 0$$

$$LP(90, 0) =$$

$$LP(52, 0) = 2$$

Keys $\rightarrow$ 10, 20, 30, 40, 50, 60, 70, 80, 90, 100

$$LP(43, 0) = 3$$

$$LP(80, 0) = 0 \text{ Collision}$$

$$LP(80, 1) = 1 \text{ Collision}$$

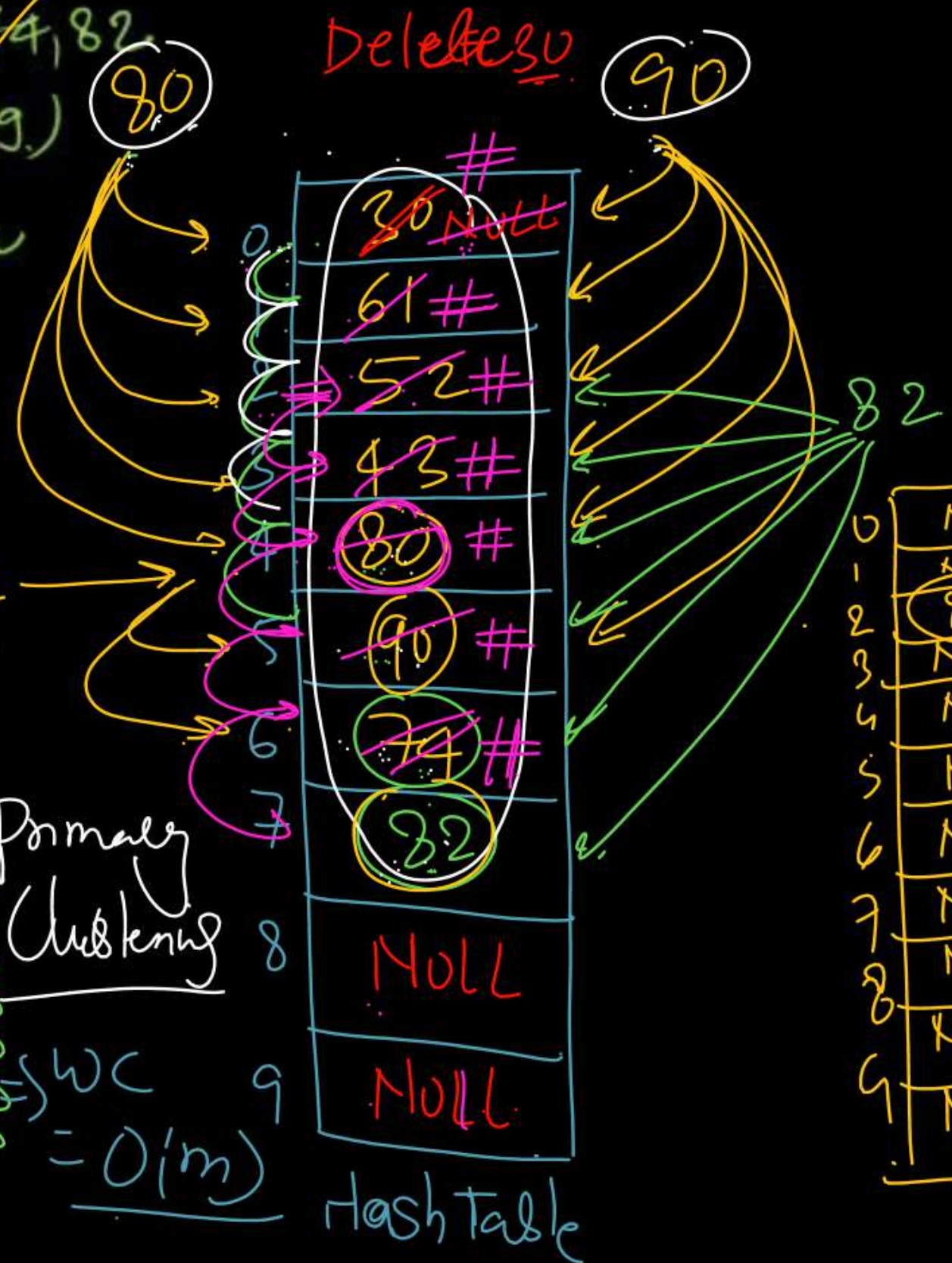
$$LP(80, 2) = 2 \text{ Collision}$$

$$LP(80, 3) = 3 \text{ Collision}$$

0	10
1	20
2	30
3	40
4	50
5	60
6	70
7	80
8	90
9	100

Primary
Clustering

SWC
= $O(m)$



Hash Table

0	NULL
1	NULL
2	82
3	NULL
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL
9	NULL

Primary Clustering:-

- 1) if two keys are mapped on the same starting location of the Hash Table then they follow the same path in the linear manner because of this the search Time Complexity will increase
- 2) To avoid this Problem the Quadratic Probing will be used
- 3) Deletion is difficult because of one element will create Trouble to other element

TC	BC	WC
Insertion	$O(1)$	$O(m)$
Searching	$O(1)$	$O(m)$
Deletion	$O(1)$	$O(m)$

NOTE:-

- 1) Deletion will create the Problem to other Keys but we will manage by storing new symbol like #
- 2) if more No. of deletion occur then Perform rehashing
- 3) Due to Primary due to Primary clustering same group as well as other group suffering But due to Secondary clustering only same group members are suffering

The keys 12, 18, 13, 2, 3, 23, 5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k) = k \bmod 10$ and linear probing. What is the resultant hash table?

(A)

0	
1	
2	2
3	23
4	
5	15
6	
7	
8	18
9	

(B)

0	
1	
2	12
3	13
4	
5	5
6	
7	
8	18
9	

(C)

0	
1	
2	12
3	13
4	2
5	3
6	23
7	5
8	18
9	15

(D)

0	
1	
2	12, 2
3	13, 3, 23
4	
5	5, 15
6	
7	
8	18
9	

0	
1	
2	12
3	13
4	2
5	3
6	23
7	5
8	18
9	15

Q. 18 Consider a hash table of size 10 with indices $\{0, 1, \dots, 9\}$, with the hash function

GATE 2025

$$h(x) = 3x \pmod{10},$$

where linear probing is used to handle collisions. The hash table is initially empty and then the following sequence of keys is inserted into the hash table: 1, 4, 5, 6, 14, 15. The indices where the keys 14 and 15 are stored are, respectively

(A) 2 and 5

(B) 2 and 6

(C) 4 and 5

(D) 4 and 6

0	
1	
2	4
3	1
4	14
5	5
6	15
7	
8	6
9	

$$\begin{array}{l} x: \quad 1 \quad 4 \quad 5 \quad 6 \quad 14 \quad 15 \\ 3x: \quad 3 \quad 12 \quad 15 \quad 18 \quad 42 \quad 45 \\ 3x \pmod{10}: \quad (3) \quad (2) \quad (5) \quad (8) \quad (2) \quad (5) \end{array}$$

A hash table of length 10 uses open addressing with hash function $h(k)=k \bmod 10$, and linear probing. After inserting 6 values into an empty hash table, the table is as shown below

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

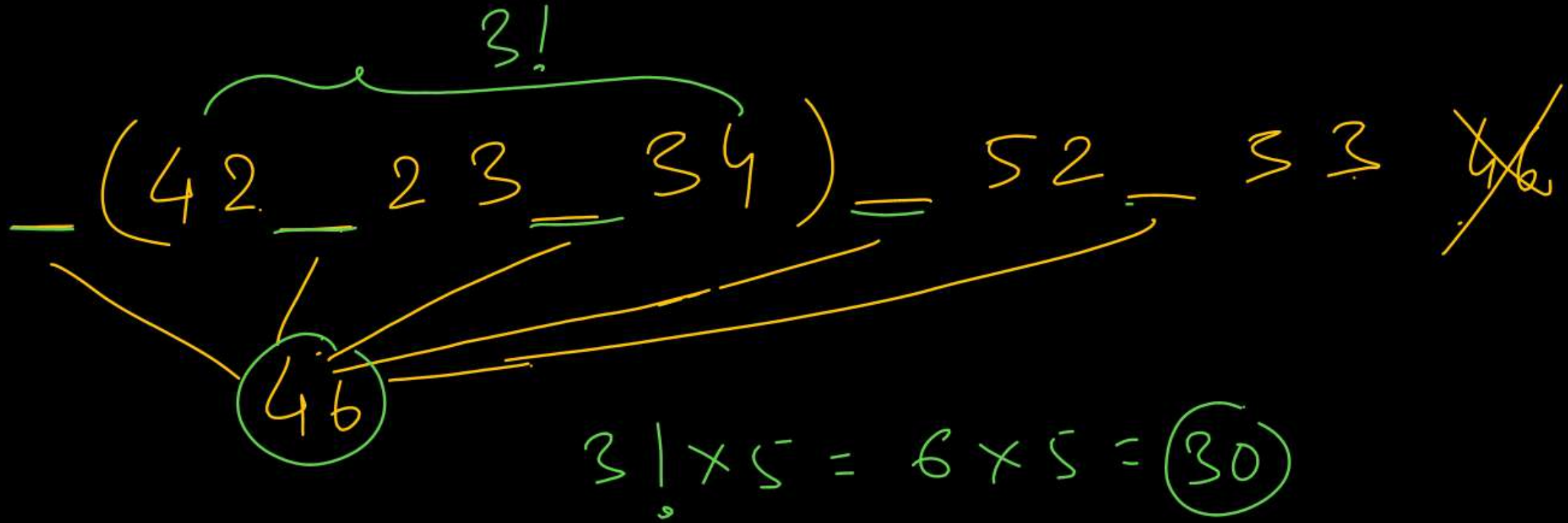
42
23
34
52
46
33

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

- A 46, 42, 34, 52, 23, 33 ✗
- B 34, 42, 23, 52, 33, 46 ✗
- C 46, 34, 42, 23, 52, 33 ✓
- D 42, 46, 33, 23, 34, 52

A hash table of length 10 uses open addressing with hash function $h(k)=k \bmod 10$, and linear probing. After inserting 6 values into an empty hash table, the table is as shown below

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	



How many different insertion sequences of the key values using the same hash function and linear probing will result in the hash table shown above?

30 Ans

Consider a hash table of size seven, with starting index zero, and a hash function $(3x + 4) \bmod 7$. Assuming the hash table is initially empty, which of the following is the contents of the table when the sequence 1, 3, 8, 10 is inserted into the table using closed hashing? Note that '-' denotes an empty location in the table.

- A** 8, -, -, -, -, -, 10
- B** 1, 8, 10, -, -, -, 3
- C** 1, -, -, -, -, -, 3
- D** 1, 10, 8, -, -, -, 3

An advantage of chained hash table (external hashing) over the open addressing scheme is

- A. Worst case complexity of search operations is less
- B. Space used is less
- C. Deletion is easier
- D. None of the above

Quadratic Probing:- $HF(Key) = Key \bmod m, m = 10 (0-9)$

$$QP(Key, i) = (HF(Key) + C_1 * i + C_2 * i^2) \bmod m$$

Keys: 25, 98, 57, 75, 97, 18, 78, 46

$C_1 = 1, C_2 = 1, i = 0, 1, 2, 3, \dots, 9 (m-1)$

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

HashTable

Ex:- $key(n) = 41, 60, 82, 93, 70, 80$

Secondary clustering:-

- 1) if two keys are mapped onto same starting location in the hash Table then they both follow the same path in the Quadratic manner because of its search time complexity will increase
- 2) Secondary clustering smaller than Primary clustering, because of this reason average searching time decreases
- 3) To avoid this problem Double Hashing will be used
- 4) Deletion will create the problem to other keys (Just like Linear Probing) But handle by special symbol (#) just like Linear Probing

TC	BC	WC
Insertion	$O(1)$	$O(m)$
Searching	$O(1)$	$O(m)$
Deletion	$O(1)$	$O(m)$

Double Hashing:- $HF_1(\text{key}) = \text{key} \bmod m$, $m = 10$ (0-9)

$$HF_2(\text{key}) = 1 + (\text{key} \bmod (m-2))$$

$$DH(\text{key}, i) = (HF_1(\text{key}) + i HF_2(\text{key})) \bmod m$$

$i = 0, 1, 2, 3, \dots, 9(m-1)$

Ex:- Keys(n) = 25, 98, 57, 75, 97, 18, 78, 65, 46 , Total Collision?

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Hash Table

Q.65 In a double hashing scheme, $h_1(k) = k \bmod 11$ and $h_2(k) = 1 + (k \bmod 7)$ are the auxiliary hash functions. The size m of the hash table is 11. The hash function for the i -th probe in the open address table is $[h_1(k) + i h_2(k)] \bmod m$. The following keys are inserted in the given order: 63, 50, 25, 79, 67, 24. The slot at which key 24 gets stored is _____. (Answer in integer)

Consider a double hashing scheme in which the primary hash function is $h_1(k) = k \bmod 23$, and the secondary hash function is $h_2(k) = 1 + (k \bmod 19)$. Assume that the table size is 23. Then the address returned by probe 1 in the probe sequence (assume that the probe sequence begins at probe 0) for key value $k=90$ is _____.

NOTE:-

1) The expected no. of Probes in an Unsuccessful Search of open Addressing Technique is $= \frac{1}{1-\alpha}$, where α is load factor

2) The expected no. of Probes in Successful search of open Addressing Technique is $= \frac{1}{\alpha} \log \frac{1}{1-\alpha}$, where α is load factor ($\alpha = \frac{n}{m}$)



 **Shailendra Singh GATE CSE/DA**

THANKYOU



 **GeeksforGeeks GATE**